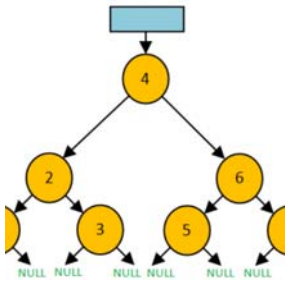


B 4.1.4 Binary Search Trees (HL Only)



B 4.1.4 Explain the structures and properties of BSTs. (HL Only)

This section explains the structure and properties of BSTs, how they organise data for $O(\log n)$ efficiency, and how to perform key operations. Learn to sketch, search, traverse, insert, and delete nodes with clear examples and diagrams. A 10-question quiz is included to test your understanding.

Introduction

Data structures are a fundamental concept in computer science, providing ways to store and organise data efficiently. While a simple list or array works for small collections, they can become slow when we need to search, add, and remove items frequently. The Binary Search Tree is a powerful data structure designed to solve this very problem, offering an elegant way to keep data sorted and accessible.

In this page you will learn about the structure of BSTs, why they are so effective for data organisation, and how to perform key operations like searching, traversing, inserting, and deleting nodes.

1. The Structures and Properties of BSTs

Let's look at the basic terminology of trees in computer science.

Tree Terminology

A **tree** is a hierarchical data structure composed of **nodes** connected by **edges**.

- **Node:** A single entity in the tree that holds a value (e.g., a number or string).
- **Edge:** A connection between two nodes.
- **Root:** The single, top-most node in a tree. It is the only node with no parent.
- **Parent:** A node that has an edge leading down to another node.
- **Child:** A node that is connected to a parent above it.
- **Leaf:** A node with no children.
- **Subtree:** A portion of the tree that can be viewed as a complete tree in itself, starting from a child node.

What is a Binary Tree?

A **Binary Tree** is a specific type of tree where every node has **at most two children**. These children are referred to as the **left child** and the **right child**.

The Binary Search Tree (BST) Property

A Binary Search Tree is a binary tree with one additional, crucial rule known as the **BST property**. This property is what makes the tree so efficient for searching.

The BST Property: For any given node **N** in the tree:

1. All values in the **left subtree** of **N** must be **less than** the value of **N**.
2. All values in the **right subtree** of **N** must be **greater than** the value of **N**.

This rule must be true for *every single node* in the tree, not just the root.

Example:

Valid BST	Invalid BST
At the root (50), everything left (30, 20, 40) is smaller, and everything right (70, 80) is larger. This holds true for every node.	The node 35 is in the left subtree of the root 40, which is correct. However, it is the right child of 30, but 35 is not greater than 30. This violates the BST property at node 30.

2. How BSTs Are Used for Data Organisation

The primary purpose of a BST is to store data in a way that combines the benefits of a sorted array and a more flexible structure like a linked list.

- **Fast Searching:** Because of the BST property, searching for an item is extremely fast. At every node, you can make a decision to go left or right, effectively eliminating half of the remaining nodes. This gives a search time complexity of $O(\log n)$ in a balanced tree, which is the same as binary search on a sorted array. In contrast, searching an unsorted list is $O(n)$, which is much slower for large datasets.
- **Efficient Insertions and Deletions:** Unlike a sorted array, where adding or removing an element might require shifting many other elements, a BST allows for insertions and deletions by just changing a few node connections (pointers). This makes it a **dynamic** data structure, well-suited for applications where data is frequently changing.

In essence, a BST organises data in a constantly sorted manner, allowing for quick lookups, additions, and removals.

3. Operations on a BST

Sketching a BST (Insertion)

To build or sketch a BST, you insert a sequence of values one at a time. The first value becomes the root. For every subsequent value, you follow a simple algorithm.

Insertion Algorithm:

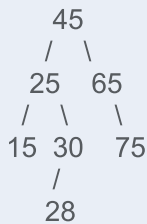
1. Start at the root node.
2. Compare the new value with the current node's value.
3. If the new value is **less**, try to move to the **left child**.

4. If the new value is **greater**, try to move to the **right child**.
5. If the chosen child node is empty (**null**), place the new node there.
6. If the chosen child node is not empty, repeat the process from step 2 with that child as the new current node.

Example: Sketch the BST for the sequence 45, 25, 65, 15, 30, 75, 28

1. **Insert 45:** Becomes the root.
2. **Insert 25:** $25 < 45$. Go left. Left is empty. Place 25 there.
3. **Insert 65:** $65 > 45$. Go right. Right is empty. Place 65 there.
4. **Insert 15:** $15 < 45$ (go left to 25). $15 < 25$ (go left). Left is empty. Place 15 there.
5. ...and so on.

The final tree would look like this:



Searching in a BST

The search algorithm is very similar to the insertion algorithm. You are essentially looking for a value until you either find it or hit an empty spot.

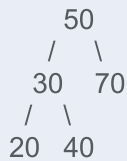
Search Algorithm:

1. Start at the root.
2. Compare the target value with the current node's value.
3. If they match, you have found the value. The search is successful.
4. If the target is **less**, move to the **left child**.
5. If the target is **greater**, move to the **right child**.
6. If the child you need to move to is empty (**null**), the value is not in the tree. The search fails.
7. Repeat from step 2 with the new current node.

Traversing a BST

Traversal means "visiting" every single node in the tree in a predefined order. There are three primary depth-first traversal methods.

Example Tree for Traversal:



1. In-order Traversal (Left, Visit, Right)

This is the most common traversal for a BST. The algorithm is:

1. Recursively traverse the left subtree.
 2. Visit the current node (e.g., print its value).
 3. Recursively traverse the right subtree.
- **Output:** 20, 30, 40, 50, 70
 - **Key Feature:** In-order traversal of a BST always produces a sorted list of its values.

2. Pre-order Traversal (Visit, Left, Right)

This traversal visits the current node before its children.

1. Visit the current node.
 2. Recursively traverse the left subtree.
 3. Recursively traverse the right subtree.
- **Output:** 50, 30, 20, 40, 70
 - **Use Case:** Useful for creating a copy of the tree.

3. Post-order Traversal (Left, Right, Visit)

This traversal visits the current node after its children.

1. Recursively traverse the left subtree.
 2. Recursively traverse the right subtree.
 3. Visit the current node.
- **Output:** 20, 40, 30, 70, 50
 - **Use Case:** Useful for safely deleting all nodes in a tree.

Deleting a Node from a BST

Deletion is the most complex operation because you must ensure the BST property is maintained after the node is removed. There are three cases to consider.

Case 1: The node to delete is a leaf (no children).

This is the easiest case. You can simply remove the node by setting its parent's reference to it to null.

- *Example: Deleting 20 from our traversal tree.*

Case 2: The node to delete has one child.

This is also straightforward. You "bypass" the node by linking its parent directly to its only child.

- *Example: If we delete 70 (and it only had a left child, say 60), we would make 50's right child point directly to 60.*

Case 3: The node to delete has two children.

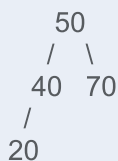
This is the most complex case.

1. **Find the In-order Successor:** This is the smallest value in the node's **right subtree**. To find it, go to the right child once, then go left as many times as possible.
2. **Copy the Value:** Copy the value from the in-order successor into the node you want to "delete".
3. **Delete the Successor:** Now, delete the original in-order successor node. This deletion will always be a Case 1 or Case 2 deletion, which we already know how to handle.

Example: Deleting node 30 from our traversal tree.

1. Node 30 has two children (20 and 40).
2. Find its in-order successor: Go right to 40. There's no left child, so 40 is the successor.
3. Copy the value **40** into the node currently holding **30**.
4. Delete the original **40** node (which is a leaf, a Case 1 deletion).

The tree now looks like this:



The BST property is maintained.

4. Summary & Key Takeaways

- A **Binary Search Tree (BST)** is a binary tree where for every node, all left descendants are less than the node, and all right descendants are greater.

- BSTs provide **$O(\log n)$** search time in a balanced tree, making them highly efficient for organizing dynamic data.
- **Insertion** involves searching for an empty spot that maintains the BST property.
- **In-order traversal** (Left, Visit, Right) visits nodes in sorted order.
- **Deletion** is handled in three cases, with the two-child case requiring the use of the **in-order successor**.

5. Knowledge Check Quiz

1

What is the defining property of a Binary Search Tree?

- A. ☐ The root node must be the smallest value in the tree.
- B. ☐ Each node can have at most two children.
- C. ☐ The tree must be perfectly balanced.
- D. ☒ For any node N, all values in its left subtree are less than N's value, and all values in its right subtree are greater than N's value.

2

If you perform an in-order traversal on a BST, what is the result?

- A. ☒ A list of the nodes in sorted, ascending order.
- B. ☐ A random ordering of the nodes.
- C. ☐ A list of the nodes in the order they were inserted.
- D. ☐ A list where parents are visited before their children.

3

Consider the following sequence of insertions into an empty BST: 50, 20, 70, 10, 30. What is the root of the tree?

A. ☐ 70

B. ☐ 10

C. ☒ 50

D. ☐ 30

4

Using the same tree from Q3 (50, 20, 70, 10, 30), where would the number 25 be inserted?

A. ☒ As the left child of 30.

B. ☐ As the right child of 30.

C. ☐ As the left child of 20.

D. ☐ As the right child of 20.

5

What is the time complexity for searching a balanced BST with 'n' nodes?

A. ☐ $O(n)$

B. ☒ $O(\log n)$

C. ☐ $O(n^2)$

D. ☐ $O(1)$

6

Which traversal visits the root node LAST?

A. ☐ In-order

B. ☒ Post-order

C. ☐ Pre-order

D. ☐ Level-order

7

You need to delete a node that is a leaf. What is the correct action?

A. ☐ Promote its parent to be a leaf.

B. ☒ Remove the node and set its parent's child reference to null.

C. ☐ Find its in-order successor.

D. ☐ Replace it with the root node.

8

You need to delete a node that has two children. What is the first step?

A. ☐ Connect the node's parent to its left child.

B. ☐ Find the smallest value in the left subtree.

C. ☐ Promote the right child to take the place of the deleted node.

D. ☒ Find the in-order successor (smallest value in the right subtree).

9

Given the tree built from 10, 5, 15, 3, 7, what is the pre-order traversal?

A. ☐ 15, 7, 3, 5, 10

B. ☐ 10, 5, 3, 7, 15

C. ☐ 3, 7, 5, 15, 10

D. ☐ 3, 5, 7, 10, 15

10

Why is a BST often preferred over a sorted array for dynamic data?

A. ☐ A BST can store more types of data.

B. ☐ Searching is always faster in a BST.

C. ☐ A BST uses less memory.

D. ☐ Insertions and deletions are generally more efficient in a BST.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**